NUMERICAL COMPUTING II
ASSIGNMENT VI
Newton Interpolation Polynomial

Complete solution report with algorithms, numerical results, plots, and reusable Python implementation.

General implementation

The program constructs Newton's divided-difference table from arbitrary distinct nodes, extracts the Newton coefficients from the first row, evaluates the polynomial using nested multiplication, and differentiates it algorithmically. The implementation does not use built-in interpolation routines or manually entered polynomial coefficients.

Newton form: $P_n(x)=f[x_0]+f[x_0,x_1](x-x_0)+\ldots+f[x_0,\ldots,x_n]\prod_{j=0}^{n-1}(x-x_j)$.

Divided differences are computed recursively:
$f[x_i,\ldots,x_{i+j}]=(f[x_{i+1},\ldots,x_{i+j}]-f[x_i,\ldots,x_{i+j-1}])/(x_{i+j}-x_i)$.

Q1. Building Newton's Polynomial

Given $f(x)=x^2+2x+1$ at $x = -2, -1, 0, 1, 2$. The data values are 1, 0, 1, 4, 9.

Divided-difference table (columns are f[x_i], first differences, second differences, and so on):

x	f[x]	1st DD	2nd DD	3rd DD	4th DD
-2	1	-1	1	0	0
-1	0	1	1	0	
0	1	3	1		
1	4	5			
2	9				

Newton coefficients are [1, 1, 1, 0, 0], so $P_4(x)=1+(x+2)+(x+2)(x+1)=x^2+2x+1$. The polynomial simplifies exactly to the original quadratic; the cubic and quartic terms have zero coefficients.

x	$P_4(x)$	Exact f(x)	Absolute error
-1.5	0.25	0.25	0
-0.5	0.25	0.25	0
0.5	2.25	2.25	0
1.5	6.25	6.25	0

Conclusion: the interpolating polynomial matches the known function, and the absolute errors are zero up to floating-point rounding.

Q2. Newton interpolation with unequally spaced data

Given $f(x)=\sin(x)$, nodes [0, 0.2, 0.55, 1.0, 1.4]. Angles are in radians.

The divided-difference table is:

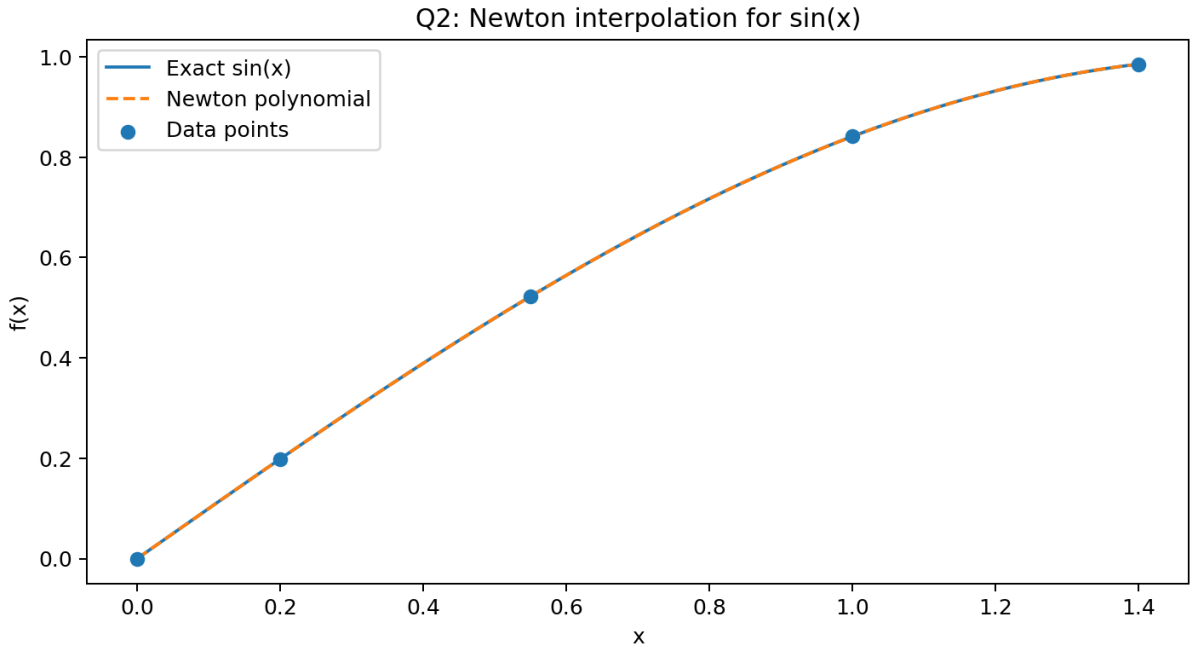
x	sin(x)	1st DD	2nd DD	3rd DD	4th DD
0	0	0.993346654	-0.122874965	-0.1488213811	0.02400435534

0.2	0.1986693308	0.9257654232	-0.2716963461	-0.1152152836	
0.55	0.5226872289	0.7084083464	-0.4099546864		
1	0.8414709848	0.359946863			
1.4	0.98544973				

Newton coefficients (rounded): 0, 0.993346654, -0.122874965, -0.1488213811, 0.02400435534

x	$P_4(x)$	sin(x)	Absolute error
0.4	0.3894673523	0.3894183423	4.901001593e-05
0.8	0.7172626697	0.7173560909	9.342116254e-05
1.2	0.932230029	0.932039086	0.0001909430221

Across the 500-point plotting grid on [0, 1.4], the maximum absolute error is 0.000209351.

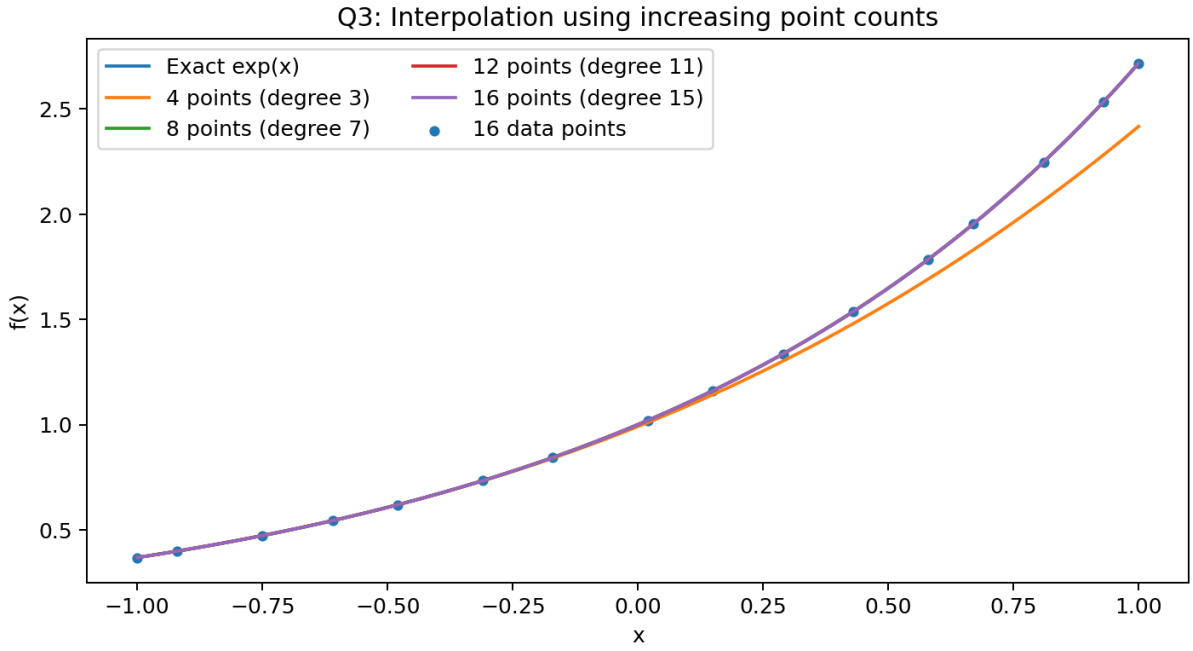


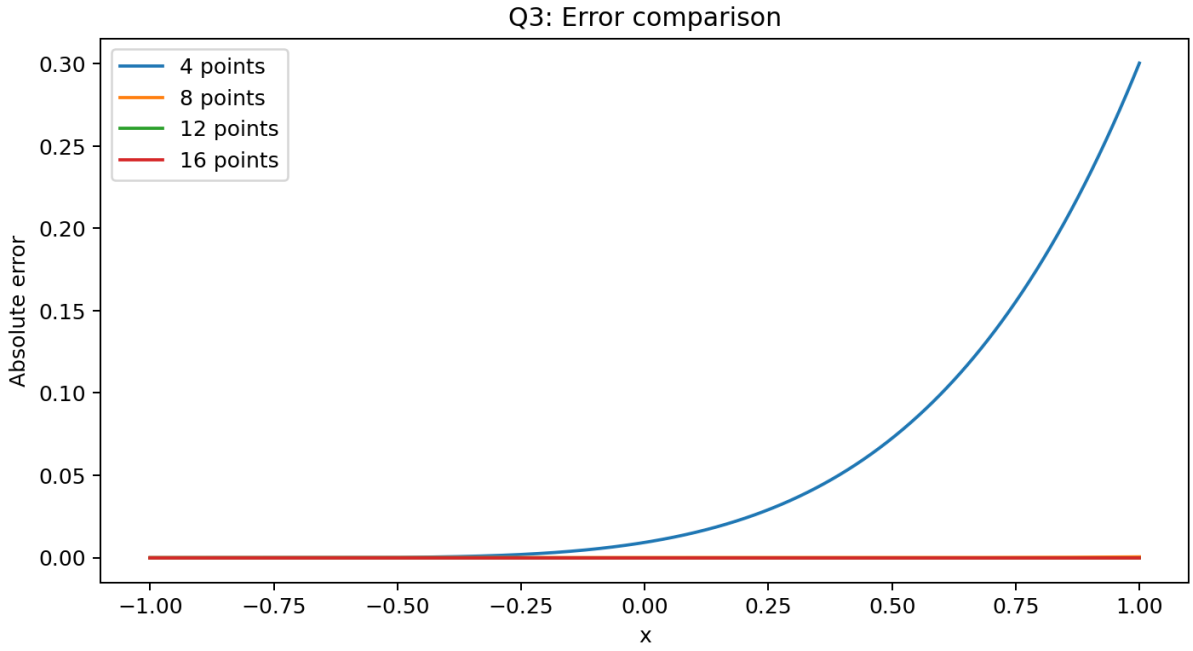
Unequally spaced nodes are allowed because divided differences use the actual node differences in their denominators. A constant-spacing forward-difference formula assumes equally spaced nodes, so it cannot be applied directly to these data without a different formulation.

Q3. Does more data always mean better interpolation?

The 16 specified nodes are used in their listed order, with $y=e^x$. For each comparison, the first 4, 8, 12, or 16 nodes form a Newton polynomial of degree at most 3, 7, 11, or 15 respectively. Errors are evaluated on 1001 uniformly distributed points over $[-1,1]$.

Number of points	Degree	Maximum absolute error	Location of maximum
4	3	0.3004602142	1
8	7	0.0004243793171	1
12	11	6.411602804e-09	1
16	15	2.109423747e-15	-0.972





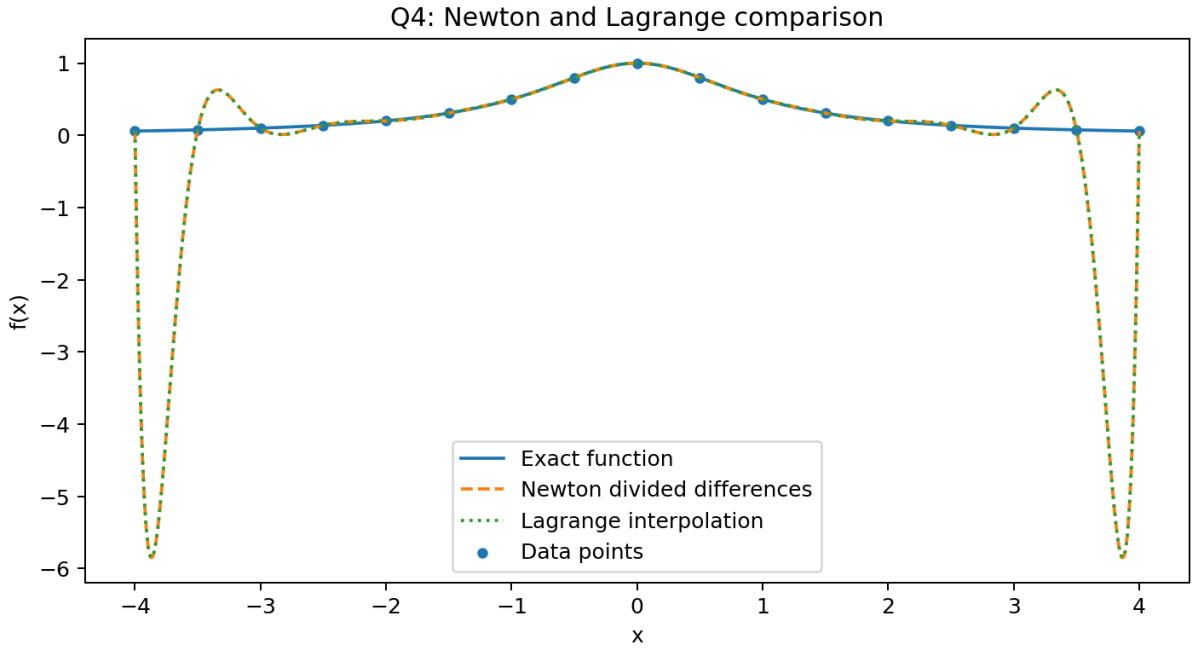
For this smooth function and these selected nodes, the table and error plot show how the approximation changes as points are added. Increasing the degree is not a universal guarantee of lower error: the result also depends on node distribution, the function, and floating-point effects. In this experiment the data are not equally spaced, so the first-n comparisons are also affected by the particular ordering of the given nodes.

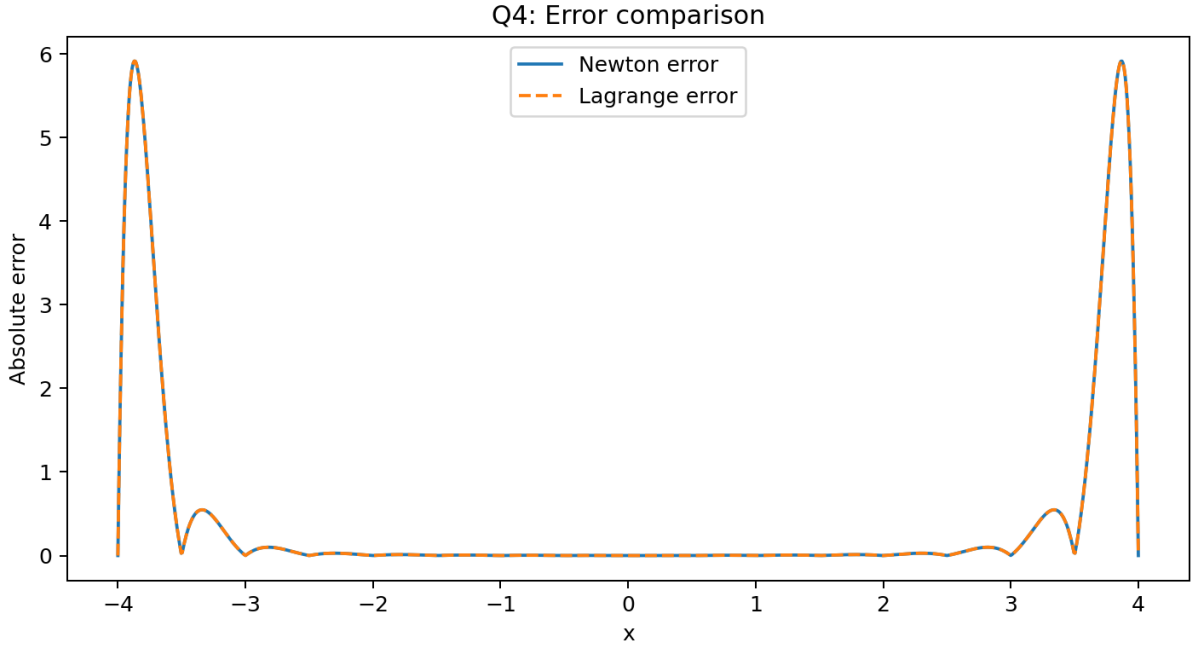
Q4. Newton vs Lagrange — are they really different?

Given $f(x)=1/(1+x^2)$ and 17 equally spaced nodes $x_i=-4+0.5i$, $i=0,\ldots,16$. Both methods use all 17 points and are evaluated on 1000 points in $[-4,4]$.

Maximum absolute error for Newton form: 5.9105839. Maximum absolute error for Lagrange form: 5.9105839.

x	Exact f(x)	Newton $P_{16}(x)$	Lagrange $P_{16}(x)$	|f−P_N|	|f−P_L|
-3.75	0.06639004149	-4.125311938	-4.125311938	4.191701979	4.191701979
-2.25	0.1649484536	0.191978874	0.191978874	0.02703042036	0.02703042036
-0.75	0.64	0.6362738985	0.6362738985	0.003726101473	0.003726101473
0.75	0.64	0.6362738985	0.6362738985	0.003726101473	0.003726101473
2.25	0.1649484536	0.191978874	0.191978874	0.02703042036	0.02703042036
3.75	0.06639004149	-4.125311938	-4.125311938	4.191701979	4.191701979





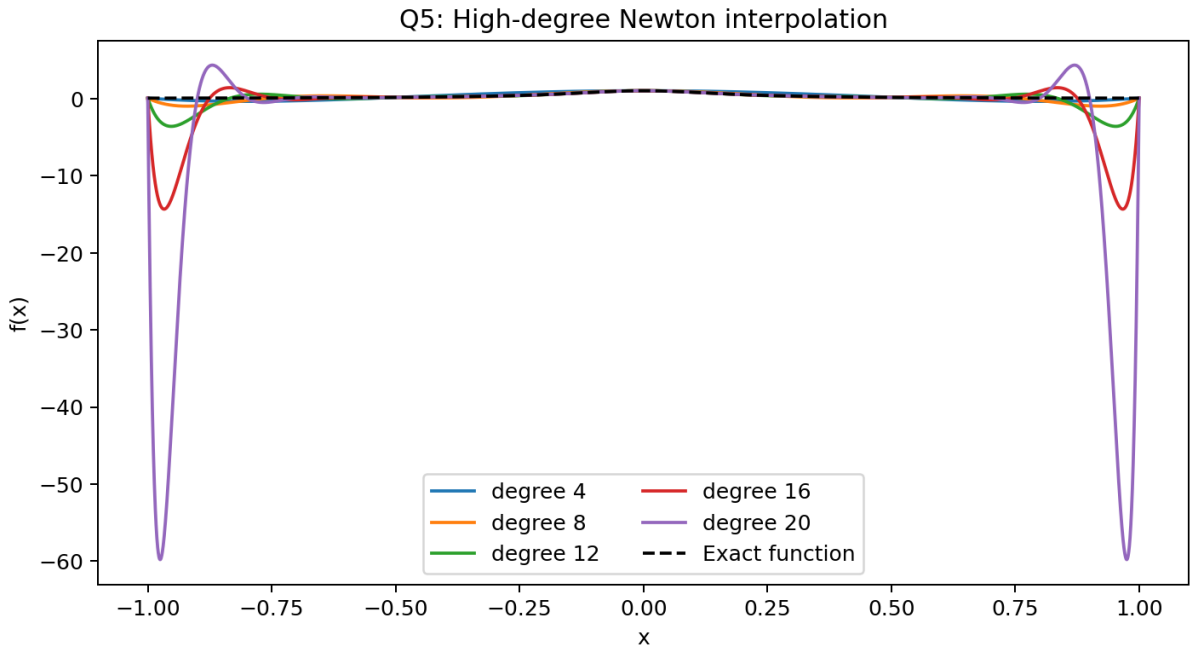
The two methods theoretically define the same unique polynomial of degree at most 16 through the same distinct nodes. Any differences in their computed values come from floating-point rounding and the numerical evaluation form. Newton form is convenient when adding a new data point because a new divided-difference coefficient can be appended; direct Lagrange form is conceptually simple but re-

evaluating it from scratch costs more operations. Both can be affected by numerical error, especially at higher degrees.

Q5. Newton interpolation and numerical error

The function is $f(x)=1/(1+25x^2)$, with equally spaced nodes $x_i=-1+2i/n$, $i=0,\ldots,n$, for $n=4,8,12,16,20$. Each polynomial is evaluated on 5001 points over $[-1,1]$.

Degree n	Number of points	Maximum absolute error	x location
4	5	0.4383571219	-0.7948
8	9	1.045173912	0.92
12	13	3.663367284	-0.9528
16	17	14.39360413	0.9672
20	21	59.82012654	-0.9748



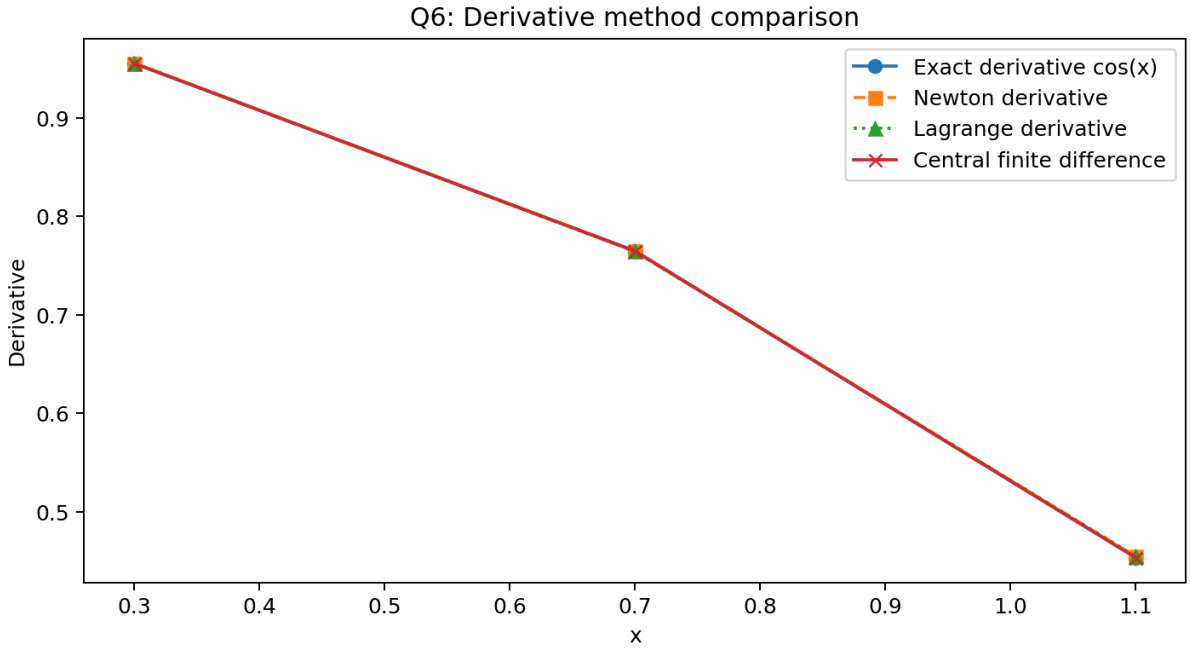
The error need not decrease as degree increases. This function is a standard illustration of the Runge phenomenon: equally spaced, high-degree polynomial interpolation can oscillate near the interval endpoints. The error is often largest near the endpoints, and higher degrees can make these oscillations worse. The experiment warns against choosing a high-degree polynomial blindly; alternatives include using better-distributed nodes (such as Chebyshev nodes) or piecewise interpolation.

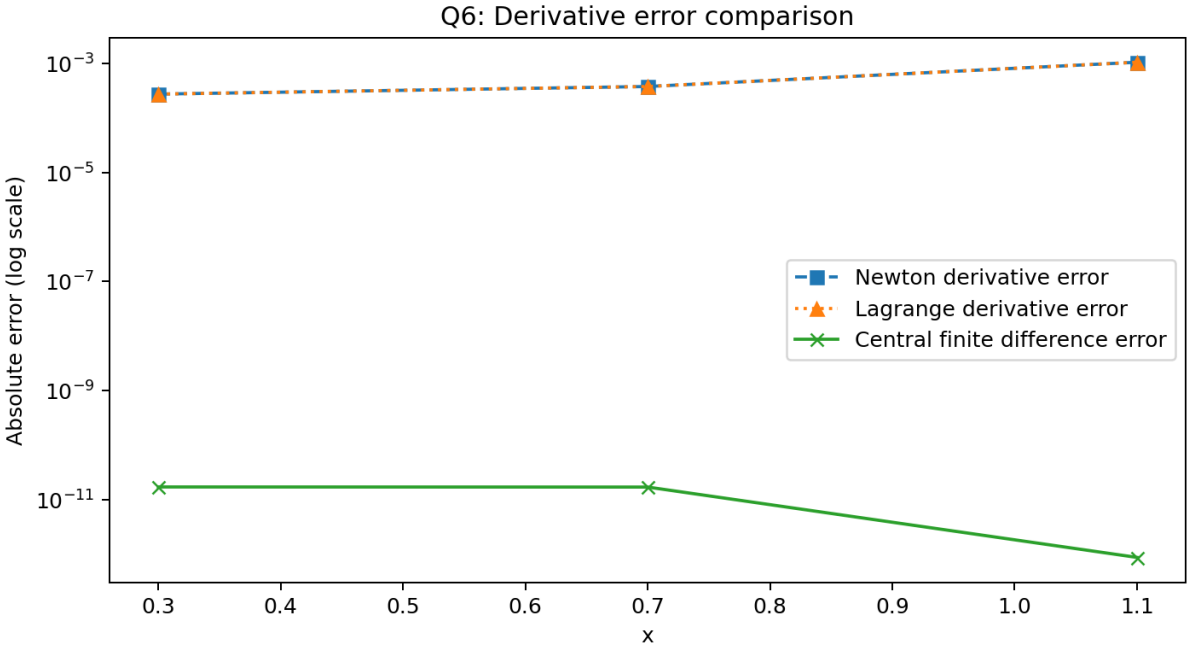
Q6. Numerical differentiation using Newton interpolation

The Newton polynomial from Q2 is differentiated algorithmically using nested multiplication. Its derivative is compared with cos(x), the exact derivative of sin(x), at x=0.3, 0.7, and 1.1. For comparison, the Lagrange polynomial is differentiated directly and a central finite difference of the known sin function is computed with $h=10^{-5}$.

x	Exact cos(x)	Newton derivative	Newton abs. error	Lagrange derivative	Lagrange abs. error	Central FD	FD abs. error
0.	0.9553364	0.9556103	0.0002738562	0.9556103	0.0002738562	0.9553364	1.71013203

3	891	454	874	454	874	891	6e-11
0.7	0.7648421873	0.7644649002	0.0003772870935	0.7644649002	0.0003772870935	0.7648421873	1.702149532e-11
1.1	0.4535961214	0.4546473193	0.0010511978 76	0.4546473193	0.0010511978 76	0.4535961214	8.731904089e-13





Because Newton and Lagrange forms represent the same interpolating polynomial, their derivative results should agree up to floating-point rounding. The central finite difference is a separate approximation that uses sin(x±h); its accuracy depends on h and round-off. Differentiating an interpolating polynomial can magnify interpolation error, so derivative accuracy need not match function-value accuracy.

Overall conclusion

Newton divided differences provide a general algorithm for interpolation with any number of distinct nodes, including unequally spaced data. Newton and Lagrange forms yield the same theoretical polynomial for the same data, although numerical evaluation can differ slightly. Increasing the number of points or polynomial degree does not always improve accuracy, particularly for equally spaced nodes and functions with sharp curvature. Numerical differentiation from an interpolation polynomial is useful but should be checked against exact derivatives when available.

Code for the assignment:

Main.py

```
import numpy as np
import matplotlib.pyplot as plt

def divided_difference_table(x, y):
    x, y = np.asarray(x, float), np.asarray(y, float)
    if len(x) != len(y) or len(np.unique(x)) != len(x):
        raise ValueError("x and y must have equal lengths and distinct x values.")
    table = np.zeros((len(x), len(x)))
    table[:, 0] = y
    for j in range(1, len(x)):
        for i in range(len(x)-j):
            table[i, j] = (table[i+1, j-1]-table[i, j-1])/(x[i+j]-x[i])
    return table

def newton_coefficients(x, y):
    return divided_difference_table(x, y)[0, :]

def newton_eval(x, c, z):
    x, c = np.asarray(x, float), np.asarray(c, float)
    z = np.asarray(z, float)
    p = np.zeros_like(z) + c[-1]
    for k in range(len(c)-2, -1, -1):
        p = c[k] + (z-x[k])*p
    return p.item() if p.ndim == 0 else p

def newton_derivative(x, c, z):
    x, c = np.asarray(x, float), np.asarray(c, float)
    p = np.zeros_like(np.asarray(z, float)) + c[-1]
    dp = np.zeros_like(np.asarray(z, float))
    for k in range(len(c)-2, -1, -1):
        dp = p + (np.asarray(z, float)-x[k])*dp
        p = c[k] + (np.asarray(z, float)-x[k])*p
    return dp.item() if np.ndim(z) == 0 else dp

def lagrange_eval(x, y, z):
    x, y = np.asarray(x, float), np.asarray(y, float)
    zz = np.asarray(z, float)
    flat, out = zz.ravel(), np.zeros(zz.size)
    for i in range(len(x)):
        basis = np.ones_like(flat)
        for j in range(len(x)):
            if i != j:
                basis *= (flat-x[j])/(x[i]-x[j])
        out += y[i]*basis
    out = out.reshape(zz.shape)
    return out.item() if np.ndim(z) == 0 else out

def lagrange_derivative(x, y, z):
    x, y = np.asarray(x, float), np.asarray(y, float)
    total = 0.0
    for i in range(len(x)):
        for m in range(len(x)):
            if i == m:
                continue
            term = 1/(x[i]-x[m])
```

```
            for j in range(len(x)):
                if j != i and j != m:
                    term *= (z-x[j])/(x[i]-x[j])
            total += y[i]*term
    return total


def report_table(name, x, y):
    table = divided_difference_table(x, y)
    print("\\n", name, "\\n", np.array2string(table, precision=8, suppress_small=True))
    print("Newton coefficients:", table[0, :])
    return table[0, :]
--------------------------------------------------------------------------------
# Q1
x1 = np.array([-2, -1, 0, 1, 2], float)
y1 = x1**2 + 2*x1 + 1
c1 = report_table("Q1 divided differences", x1, y1)
test1 = np.array([-1.5, -0.5, 0.5, 1.5])
p1 = newton_eval(x1, c1, test1)
print("Q1 values and errors:", np.c_[test1, p1, np.abs(test1**2+2*test1+1-p1)])
 ----------------------------------------------------------------
# Q2
x2 = np.array([0, 0.2, 0.55, 1.0, 1.4])
y2 = np.sin(x2)
c2 = report_table("Q2 divided differences", x2, y2)
for z in [0.4, 0.8, 1.2]:
    p = newton_eval(x2, c2, z)
    print("Q2", z, p, np.sin(z), abs(np.sin(z)-p))
grid = np.linspace(0, 1.4, 500)
plt.figure(); plt.plot(grid, np.sin(grid), label="sin(x)")
plt.plot(grid, newton_eval(x2, c2, grid), "--", label="Newton")
plt.scatter(x2, y2, label="data"); plt.legend(); plt.title("Q2 interpolation");
plt.savefig("Q2_interpolation.png", dpi=160)
plt.figure(); plt.plot(grid, np.abs(np.sin(grid)-newton_eval(x2, c2, grid)))
plt.title("Q2 absolute error"); plt.savefig("Q2_error.png", dpi=160)
--------------------------------------------------------------------------
# Q3
x3 = np.array([-1.0,-0.92,-0.75,-0.61,-0.48,-0.31,-0.17,0.02,0.15,0.29,0.43,0.58,0.67,0.81,0.93,1.0])
grid3 = np.linspace(-1, 1, 1001)
print("\\nQ3 maximum errors")
for m in [4, 8, 12, 16]:
    xx, yy = x3[:m], np.exp(x3[:m])
    cc = newton_coefficients(xx, yy)
    err = np.abs(np.exp(grid3)-newton_eval(xx, cc, grid3))
    print(m, m-1, err.max(), grid3[np.argmax(err)])
-----------------------------------------------------------------------------
# Q4
x4 = np.linspace(-4, 4, 17); y4 = 1/(1+x4**2)
c4 = newton_coefficients(x4, y4)
grid4 = np.linspace(-4, 4, 1000)
pn, pl = newton_eval(x4, c4, grid4), lagrange_eval(x4, y4, grid4)
print("\\nQ4 max errors:", np.max(np.abs(1/(1+grid4**2)-pn)),
      np.max(np.abs(1/(1+grid4**2)-pl)))
plt.figure(); plt.plot(grid4, 1/(1+grid4**2), label="exact")
plt.plot(grid4, pn, "--", label="Newton"); plt.plot(grid4, pl, ":", label="Lagrange")
plt.legend(); plt.title("Q4 comparison"); plt.savefig("Q4_comparison.png", dpi=160)
-------------------------------------------------------------------------------------
# Q5
f5 = lambda z: 1/(1+25*np.asarray(z)**2)
grid5 = np.linspace(-1, 1, 5001)
print("\\nQ5 maximum errors")
for n in [4, 8, 12, 16, 20]:
    xx = -1 + 2*np.arange(n+1)/n
    cc = newton_coefficients(xx, f5(xx))
    err = np.abs(f5(grid5)-newton_eval(xx, cc, grid5))
    print(n, n+1, err.max(), grid5[np.argmax(err)])
-----------------------------------------------------------------------------------
# Q6
print("\\nQ6 derivative comparison: x, exact, Newton, Lagrange, central FD")
for z in [0.3, 0.7, 1.1]:
    nd = newton_derivative(x2, c2, z)
    ld = lagrange_derivative(x2, y2, z)
    h = 1e-5
    fd = (np.sin(z+h)-np.sin(z-h))/(2*h)
    print(z, np.cos(z), nd, ld, fd)
plt.show()
```